

Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title:** Script-Controlled ACL – Restrict Record Access Based on Field Value
- **Team ID:** SWTID-2026-4457 (date: 01 Oct 2026)
- **Team Members:** List of team members and their roles.

Name	Role
K.Dharshini	Backend
A.Nagasundhari	Frontend
U.Bharathi & S.Hemalatha	Database

2. Project Overview

- **Purpose:** The project demonstrates how to implement a script-controlled Access Control List (ACL) on the ServiceNow platform so that users can view records only when they meet specific conditions. In this scenario, only users who belong to the EEE branch (role bb1) can view records whose Branch field is EEE, users without the role see nothing, and administrators retain full access. Further ACLs control the create, write and delete operations through separate roles.
- **Features:**
 - Custom table **Institution Details** (`u_institution_details`) holding student and faculty information.
 - Test user (EEE User) and four custom roles: **bb1** (read), **bb2** (create), **bb3** (write), **bb4** (delete).
 - Script-controlled **Read ACL** with the data condition *Branch is EEE* and an admin override.
 - Separate **Create, Write and Delete ACLs**, each tied to its own role.
 - Verification by impersonating the EEE user, a user without the role, and the administrator.

3. Architecture

- **Frontend:** There is no React frontend. The user interface is the standard ServiceNow UI: the Application Navigator (All menu), the list view `u_institution_details.list`, the record form, inline list editing, the New button and the Actions on selected rows menu. The same ACLs also apply in Service Portal interfaces. What each user sees (rows, New button, editable cells, delete action) is decided by the ACLs.
- **Backend:** There is no Node.js / Express.js server. The backend is the ServiceNow platform (application scope: Global, update set “ACL Project”). Every request to read, create, write or delete a record is evaluated on the server by the ACLs on `u_institution_details`. The Read ACL is Advanced: it requires the bb1 role, applies the data condition *Branch is EEE*, and runs a server-side script that calls `gs.hasRole()`.

- **Database:** There is no MongoDB. Data is stored in the ServiceNow table `u_institution_details` (label: Institution Details, extends: none). The schema is shown below; the sample data contains six records spread over the ECE, EEE and CSE branches.

Field (label)	Type	Notes
Student Roll Number	String / Auto Number	Default value <code>javascript:global.getNextObjNumberPadded()</code> ; (e.g. INS0001007)
Student Name	Reference (User)	Max length 32
Faculty name	Reference (User)	Max length 32
Branch	Choice (ECE, EEE, CSE)	Dictionary shows String, max length 40
Email	String	Max length 40
Phone Number	String	Max length 40
Description	Multi-line text	Max length 1,000

Interactions: the ACLs sit between the user's request and the table. A `bb1` user listing the table gets only the 2 EEE rows, a user without the role gets an empty list, and an admin gets all 6 rows.

4. Setup Instructions

- **Prerequisites:**
 - A ServiceNow instance with administrator access (the lab used an instance with application scope Global).
 - Ability to elevate the **security_admin** role (needed to create or edit ACLs).
 - A modern web browser. Node.js and MongoDB are not required.
- **Installation:** There is nothing to clone or install and no environment variables. Configure the instance as follows.
 - **Step 1 – User:** User Administration → Users → New. User ID: EEE User, First name: EEE, Last name: User, Email: `eeeuser@gmail.com`. Save.
 - **Step 2 – Roles:** User Administration → Roles → New. Create **bb1**, **bb2**, **bb3**, **bb4** and assign all four to the EEE User.
 - **Step 3 – Table:** System Definition → Tables → New. Label: Institution Details, Name: `u_institution_details`, Extends: false. Add the fields listed in Section 3 and save.
 - **Step 4 – Data:** insert several records with different Branch values (ECE, EEE, CSE).
 - **Step 5 – Elevate:** profile menu → Elevate role → select `security_admin` → Update.
 - **Step 6 – Read ACL:** System Security → Access Control (ACL) → New. Type: record, Operation: read, Name: `u_institution_details`, Active: true, Advanced: true. Requires role: `bb1`. Data condition: Branch is EEE. Add the script given in Section 7 and submit.
 - **Step 7 – Create ACL:** Operation: create, same table, Requires role: `bb2`, no data condition.
 - **Step 8 – Write ACL:** Operation: write, same table, Requires role: `bb3`, no data condition.
 - **Step 9 – Delete ACL:** Operation: delete, same table, Requires role: `bb4`, no data condition.

5. Folder Structure

- **Client:** The UI layer has no source folders. It is organised by the navigation modules and views used in the project:

```
Client (ServiceNow UI)
|- All menu -> User Administration
|   |- Users (EEE User)
|   |- Roles (bb1, bb2, bb3, bb4)
|- All menu -> System Definition -> Tables (u_institution_details)
|- All menu -> System Security -> Access Control (ACL)
'- List view: u_institution_details.list
```

- **Server:** The server side is the set of configuration records stored on the instance (update set “ACL Project”, scope Global):

```
Server (ServiceNow instance)
|- sys_user EEE User
|- sys_user_role bb1, bb2, bb3, bb4
|- sys_db_object u_institution_details
|- sys_dictionary 7 custom columns
'- sys_security_acl read (bb1 + script), create (bb2),
write (bb3), delete (bb4)
```

6. Running the Application

- There is no server to start; the instance is hosted. To run and check the solution:
 - **Frontend:** impersonate a user (profile menu → Impersonate user), then type `u_institution_details.list` in the All menu filter to open the Institution Details list.
 - **Backend:** nothing to run. The ACLs are evaluated automatically on every list, form and update action.
 - Repeat as the EEE User, a user without the roles, and the administrator, and compare the lists.

7. API Documentation

- The project does not add custom REST endpoints. The access “endpoints” are the four record ACLs on `u_institution_details`:

Operation	Requires role	Data condition	Script	Behaviour
read	bb1	Branch is EEE	Yes (Advanced)	bb1 user sees only EEE rows; admin sees all
create	bb2	None	No	New button available
write	bb3	None	No	Cells and fields can be edited
delete	bb4	None	No	Delete action available

- Script used in the Read ACL:

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }
  // Deny access for all others
  return false;
})();
```

- Example responses (as observed in the list view): bb1 user – 2 records (INS0001007, INS0001004); user without the role – “No records to display”; admin – 6 records.
- The platform’s standard REST Table API for this table is expected to honour the same ACLs; it was not tested in this project.

8. Authentication

- Authentication is handled by ServiceNow’s standard instance login and user session. No custom tokens or session code are used.
- Authorization is handled by roles and ACLs. A user’s roles are checked by the ACLs (Requires role) and by the script (`gs.hasRole()`).
- Changing ACLs requires the elevated **security_admin** role, which lasts only until the user logs out.
- The **admin** role is explicitly allowed in the read script, and each ACL has *Admin overrides* checked.

User	Roles	Can do
Administrator	admin (security_admin when elevated)	View all 6 records; manage ACLs
EEE User	bb1, bb2, bb3, bb4	View EEE records only; create, edit and delete them
User without the roles	none	No records displayed

9. User Interface

- Screenshots of the list view for each type of user and operation:

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE		saiguna@gmail.com	Abel Tutor	9090898765	Guna Sai
INS0001004	EEE		prashanth@gmail.com	Abel Tutor	9090909090	Prashanth Reddy

Figure 1: User with bb1 sees only EEE branch records

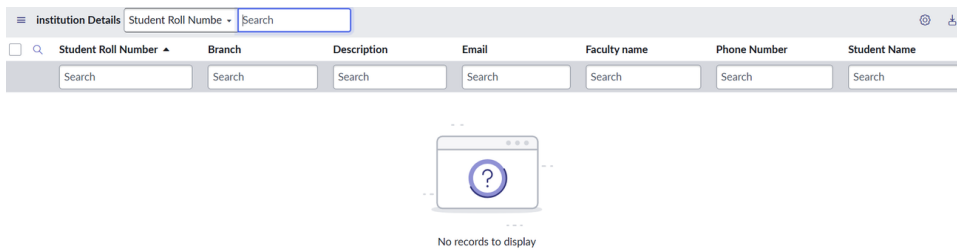


Figure 2: User without the role sees no records

Figure 3 shows an administrator seeing all records regardless of branch. The interface includes a search bar and a table with columns: Student Roll Number, Faculty name, Student Name, Branch, Email, and Phone Number. A "New" button is visible in the top right corner.

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

Figure 3: Administrator sees all records regardless of branch

Figure 4 shows a user seeing EEE records and the New button. The interface includes a search bar and a table with columns: Student Roll Number, Branch, Description, Email, Faculty name, Phone Number, and Student Name. A "New" button is visible in the top right corner. A red box highlights the "Student Roll Number" and "Branch" columns.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE		saiguna@gmail.com	Abel Tuter	9090898765	Guna Sai
INS0001004	EEE		prashanth@gmail.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 4: bb1 + bb2 user sees EEE records and the New button

Figure 5 shows a user editing the Description cell. The interface includes a search bar and a table with columns: Student Roll Number, Branch, Description, Email, Faculty name, Phone Number, and Student Name. A "New" button is visible in the top right corner. A red box highlights the "Description" column.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE		saiguna@gmail.com	Abel Tuter	9090898765	Guna Sai
INS0001004	EEE		prashanth@gmail.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 5: bb1 + bb2 + bb3 user can edit the Description cell

Figure 6 shows a user using the Delete action. The interface includes a search bar and a table with columns: Student Roll Number, Branch, Description, Email, Faculty name, Phone Number, and Student Name. A "New" button is visible in the top right corner. A red box highlights the "Delete" button in the actions menu.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE	ACL is Executed	saiguna@gmail.com	Abel Tuter	9090898765	Guna Sai
INS0001004	EEE	ACL is Executed	prashanth@gmail.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 6: bb1–bb4 user can use the Delete action

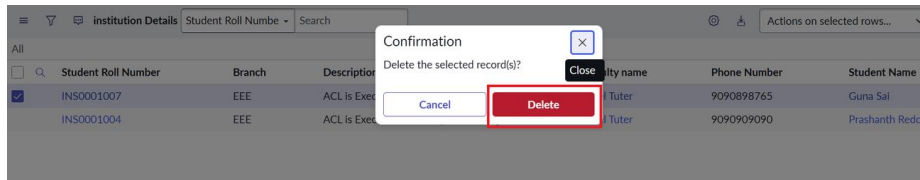


Figure 7: Delete confirmation dialog

10. Testing

- Strategy:** Manual, role-based functional testing. Each ACL was tested right after it was created by impersonating a user who has the relevant roles and comparing the list view with the expected result. Tools: ServiceNow Impersonate user, Elevate role and the list view `u_institution_details.list`.

ID	Scenario	Expected	Actual	Status
TC-1	User with bb1 opens the list	Only EEE records (2)	2 EEE records shown	Pass
TC-2	User without bb1 opens the list	No records	"No records to display"	Pass
TC-3	Admin opens the list	All records (6)	6 records shown	Pass
TC-4	bb1 + bb2 user views the list	EEE rows and New button	2 rows, New button visible	Pass
TC-5	bb1 + bb2 + bb3 user edits Description	Inline edit allowed	Description editable	Pass
TC-6	bb1–bb4 user deletes a record	Delete action available	Delete option and dialog shown	Pass

Result: 6 of 6 test cases passed.

11. Screenshots or Demo

- Configuration screenshots from the project:

Figure 8: Creation of the test user (EEE User)

Figure 9: Creation of role bb1 (bb2, bb3 and bb4 follow the same procedure)

Table **Institution Details** Delete Update Delete All Records

A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. [More Info](#)

* Label **Institution Details** Application **Global**

* Name **u_institution_details** Remote Table ☐

Figure 10: Custom table Institution Details (u_institution_details)

Table Columns for text Search 1 to 13 of 13

Dictionary Entries

Column label	Type	Reference	Max length	Default value
Updates	Integer	(empty)	40	
Created by	String	(empty)	40	
Student Roll Number	String	(empty)	40	javascript:global.getNextObjNumberPadded();
Description	String	(empty)	1,000	
Updated by	String	(empty)	40	
Phone Number	String	(empty)	40	
Sys ID	Sys ID (GUID)	(empty)	32	
Created	Date/Time	(empty)	40	
Faculty name	Reference	User	32	
Branch	String	(empty)	40	
Updated	Date/Time	(empty)	40	
Student Name	Reference	User	32	
Email	String	(empty)	40	

Insert a new row...

Figure 11: Table columns (dictionary entries)

Institution Details Student Roll Number Search Actions on selected rows...

All

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
Search	Search	Search	Search	Search	Search
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

Figure 12: Sample records with ECE, EEE and CSE branches

Elevate role

Close dialog

Elevate a role by adding privileges, which end when you log out. [Learn more](#)

AVAILABLE ROLES

☒ security_admin

Grant modification access to High Security Settings, allow user to modify the Access Control List

Cancel

Update

Figure 13: Elevating the security_admin role

Access Control
u_institution_details

* Type: record

* Operation: read

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: No. of records matching the condition: 6

Application: Global

Active: ☒

Advanced: ☒

Buttons: Add Filter Condition, Add OR Clause

Fields: -- choose field --, -- oper --, -- value --

Figure 14: Read ACL – type, operation, name and Advanced option

Data Condition

Condition: No. of records matching the condition: 2

Branch is EEE

Buttons: AND, OR, X

Advanced Condition

Controlled by References: ☐

Script: ☒ Turn on ECMAScript 2021 (ES12) mode

```

1
2
3
4
5
6
7
8
(function() {
  if (gs.hasRole('bb1')) {
    return true;
  } else {
    return false;
  }
})();

```

Figure 15: Data condition (Branch is EEE) and the ACL script

Access Control
u_institution_details

* Type: record

* Operation: create

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: No. of records matching the condition: 6

Application: Global

Active: ☒

Advanced: ☐

Buttons: Add Filter Condition, Add OR Clause

Fields: -- choose field --, -- oper --, -- value --

Figure 16: Create ACL

Conditions

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role	Role
X	BB2
+	Insert a new row...

Security Attribute Condition

Figure 17: Create ACL requires role bb2

Access Control
u_institution_details

* Type: record

* Operation: write

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: No. of records matching the condition: 6

Requires role:

Role
BB3

Insert a new row...

Figure 18: Write ACL

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role

Role
BB3

Insert a new row...

Figure 19: Write ACL requires role bb3

Access Control
New record

Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.

* Type: record

* Operation: delete

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: No. of records matching the condition: 6

Requires role:

Role
BB4

Insert a new row...

Figure 20: Delete ACL

Requires role

Role
BB4

Insert a new row...

Security Attribute Condition

Figure 21: Delete ACL requires role bb4

- Demo link: _____

12. Known Issues

- The read script returns true for any user with bb1; the restriction to EEE records comes from the data condition, so the two must be kept in sync.
- Create, write and delete ACLs have no data condition, so they are not limited by branch on their own.
- Only the EEE branch is covered. Supporting ECE and CSE needs additional roles and ACLs.
- The Branch column appears as type String in the dictionary screenshot, although the design calls for a Choice field (ECE, EEE, CSE).

- The script printed in the instructions (with an admin check) differs from the shorter script visible in the ACL screenshot (bb1 check only). Use the full script so that administrators are covered explicitly.

13. Future Enhancements

- Add roles and ACLs for the ECE and CSE branches, ideally driven by a user attribute instead of one role per branch.
- Add data conditions to the create, write and delete ACLs so changes are also branch-limited.
- Add field-level ACLs to protect Email and Phone Number.
- Use groups or containing roles to simplify role assignment.
- Add audit reports for access reviews and publish the records through a Service Portal page.